

Figure 1: A sample plot showing the closing index value and the difference between the day's high and low values

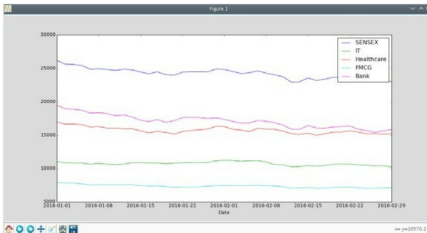


Figure 2: A sample plot showing the closing index values for several indices

```
# Date is implicitly included as it is the index column
df2 = df[['Index', 'Close', 'Volatility']]
# list the entries for Jan 1, 2016
print(df2['2016-01-01'])
# save the dataframe as a csv file
df2.to_csv('BSE_indices.csv')
```

Plotting data from a dataframe

An attraction of the dataframe object is that you can select parts of data very easily. Try the following:

```
>>> import pandas as pd
>>> import matplotlib.pyplot as plt
>>> df = pd.read_csv('BSE_indices.csv', index_col='Date')
>>> bank=df['Index']=='Bank'
>>> df[bank].plot(figsize=(15,7), subplots=True)
>>> plt.legend(loc='best')
```

```
>>> plt.show()
```

In the above code snippet, you have selected the rows in which the index is *Bank*, and plotted the *Close* and *Volatility* columns as sub-plots. The result is shown in Figure 1.

Reorganising data

As you might have discovered, it is easy to select the rows you want and add columns. So if you want to organise the data such that each index corresponds to a column, you can do so using the following code:

```
import pandas as pd
def select_index(df, index):
    # Select rows for a specific index
    s = df['Index'] == index
    # return a dataframe with selected rows
    return df[s]

df = pd.read_csv('BSE_indices.csv',
index_col='Date')
# the unique values of Index column
indices=set(df['Index'])
# Create an empty dataframe
df_col = pd.DataFrame()
for index in indices:
    # add the closing value as a column
    df_col[index] = select_
index(df, index)['Close']
# plot the indices
```

```
import matplotlib.pyplot as plt
df_col.plot(figsize=(15,7))
plt.legend(loc='best')
plt.show()
```

Select the unique index names. Then, for each index, select only the desired rows and add the closing value column to a dataframe as a new column. Finally, plotting the data is easy. The result is shown in Figure 2.

Hopefully, you are convinced that if you need to make sense of data from various sources, Python-Pandas can be an immensely valuable tool. 

By: Dr Anil Seth

The author has earned the right to do what interests him. You can find him online at <http://sethanil.com> and <http://sethanil.blogspot.com>, or you can reach him via email at anil@sethanil.com.